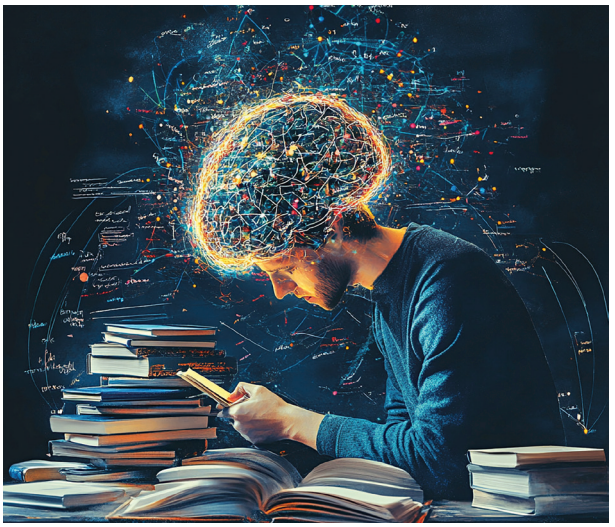


rent laptop warranty procedure?" needs a company document. "Which exception did we agree to make for this customer?" may require a stored conversation or authorised case record. Treating these as identical questions creates errors and potential privacy problems.

Our project is a local document assistant. It will accept a question, find relevant excerpts from the cleaned files prepared in Chapter 2, pass those excerpts to the existing local model and return an answer with references. Keep your original files and metadata intact; they remain the source of truth.

Follow one question through RAG

The pipeline is easy to draw:

Documents → chunks → embeddings → searchable index

Question → retrieval → supporting passages → model → cited answer

The first line is *indexing*: preparation performed when documents are added or changed. The second is *querying*: work performed when someone asks a question. Separating them avoids redundant processing.

Suppose the question is, "Can I return an opened laptop?" Search should find the return-policy clause, including exceptions and dates, before the model starts drafting. The final prompt might contain the user question, three policy excerpts and instructions to answer only from those excerpts. If none addresses opened laptops, the correct response is uncertainty, not an invented seven-day rule.

RAG also creates new failure points. Extraction might omit a table; a chunk could separate the exception from the rule; search might return an obsolete policy; or the model might embellish an otherwise accurate passage. We will test each stage rather than judging the fluent answer alone.

Build your first local RAG pipeline

We will use Python, Sentence Transformers and Chroma. Chroma persists the search collection on disk; Sentence Transformers creates the embeddings. For generation, the example calls an Ollama-compatible local endpoint. If Part 1 left you using another

